

Lifelong Learning Techniques

for an Origami Robot

by

E. De Vroey

Student number: 4685156

Contents

1	Introduction	1
1.1	Application to soft (origami) robots	1
1.2	Challenges	1
2	Background and definition	3
3	Blackbox methods	5
3.1	Learning control policies	5
3.2	Learning the dynamic model	6
3.3	Incremental learning.	7
3.4	Summary	8
4	Learning architectures	9
4.1	Dual architectures	9
4.2	Architectural expansions	9
5	Storage and retrieval	11
6	Discussion and conclusion	13

1

Introduction

1.1. Application to soft (origami) robots

1.2. Challenges

(Kim2021; Lesort2020).

How can lifelong learning methods be employed to manage the evolving dynamics due to wear and tear in the control of a soft origami robot over its operational lifespan?

2

Background and definition

(Lesort2020; Landau2011; Annaswamy2021).

3

Blackbox methods

The term ‘blackbox’ in the context of lifelong learning techniques describes models that are inherently ‘data-driven’, requiring vast datasets to predict outcomes while often lacking in interpretability. These methods differ from traditional approaches, where clever architectural approaches or intuitive mathematical tricks might be exploited. Instead, so-called blackbox methods trade transparency for performance, as they process and learn from information in hard-to-interpret yet effective ways.

In this chapter, an overview is provided of how blackbox methods can be used as either building blocks for lifelong learning or to solve the lifelong learning challenge as complete solutions.

3.1. Learning control policies

A perfect example of a blackbox method is given by **Alessi2023<empty citation>**. In their study they used Proximal Policy Optimization (PPO) (**Schulman2017**) to obtain a feedforward neural network (FNN) that can generalize over new dynamics and tasks. The controller is a blackbox: a desired trajectory, several measurements of the robots state, and other values of interest, such as the error of robot’s end-effector tip, are used as inputs, resulting in a control action. It essentially acts in a manner akin to a feedback controller using an inverse dynamics model, only in this case it is unknown what is happening *under the hood*.

The previous example is “*perfect*” in the sense that it perfectly embodies the simplistic nature of blackbox methods, however it is worth noting that other techniques, such as more complicated architectures or combinations of multiple models, can be used in combination with these blackbox methods. Such architectural modifications will be discussed in chapter 4.

Apart from **Alessi2023<empty citation>**, several other examples of blackbox policies can be found in the literature.

In **Zhou2022<empty citation>**, two policies are learned for the control of a small humanoid robot. Exact details on the inputs and outputs of the controllers are not provided, but both policies are probabilistic in nature.

Thuruthel2019<empty citation> use guided policy search (**Levine13**) over trajectory samples containing control actions for given regions in the state-space of a soft robotic manipulator. The resulting control policy is represented as a FNN which takes in current and previous states, as well as a desired target state, and returns the appropriate control action.

For the control of a soft robotic arm, in **Nazeer2023<empty citation>**, the policy is represented by a multi-layer perceptron (MLP) which takes in the arm’s current state¹ and outputs

¹Rather, the sum of the state and a correction term, resulting from a separate network (section 4.2), but that is not

a control action. The policy is trained using the Soft Actor-Critic (SAC) (Haarnoja2018) reinforcement learning algorithm.

In Lu2020<empty citation>, a more complicated technique is used. A probabilistic ‘skill’ distribution $p_\psi(z | s)$ and a neural network-based policy $\pi_\theta(a | s, z)$ are independently and simultaneously learned using SAC. Both models are trained on rollouts of state transitions generated by a dynamics model. However, once learned, a model-based planning algorithm called Model Path Predictive Integral (MPPI) (Williams2015) is used to find the optimal skill sequence in the skill distribution p_ψ , resulting in executable actions through the policy π_θ .

3.2. Learning the dynamic model

One way of having continually adapting control for soft robots, is by implementing a (traditional) control scheme that uses a dynamic model (either forward or inverse) of the robot and then to incrementally update that model. The idea of learning a dynamic model for soft robots has advantages even outside the context of lifelong adaptation: many soft robots have such a large amount of degrees of freedom and such complex and nonlinear dynamics, that analytically deriving a dynamic model is both cumbersome and often insufficiently accurate (Kim2021; Thuruthel2019; Giorelli2015; Gillespie2018).

Blackbox methods also allow for learning both forward and inverse dynamic models and can thus be used in several control schemes. When learning a forward model, the robot’s actions and resulting states can be used as the inputs and outputs, respectively, this is so-called *direct modeling* (NguyenPeters2011). In the case of learning inverse dynamic models, such direct approaches might not always be applicable, due to the multiple solutions that exist for the inverse dynamics of robots with redundant degrees of freedom. Instead, an *indirect-modeling* approach can be taken, where the model is trained to minimize the error of the feedback controller (NguyenPeters2011).

The form that the dynamic model takes can vary. For example, in Gillespie2018<empty citation>, a neural network is trained to predict the next state of the robot, given the current state and a control action, the analytical gradients of these outputs w.r.t. to the inputs are made available during the process of backpropagation and can be used in matrices A and B to form a more interpretable state-space system. In Thuruthel2019<empty citation> on the other hand, a forward model is learned for open-loop control, using the control action, the current state, and the previous state as input, and the next state as output to train a recurrent neural network. This forward model is kept in its more abstract blackbox form as a recurrent neural network (RNN), and is used in combination with trajectory optimization to generate trajectories in open-loop control.

Since the lifelong adapting nature of the techniques is embedded in the dynamic model, the choice of control scheme in which the learned dynamic model is used can vary from implementation to implementation. To illustrate, the aforementioned matrices A and B from Gillespie2018<empty citation> are used in a model predictive control setting, while the generated trajectories from Thuruthel2019<empty citation> are used as samples to train a blackbox control policy (the dynamic model is in essence an intermediary step in order to create the blackbox policy mentioned in section 3.1). More examples several blackbox dynamic models and their control scheme can be found in the literature.

Nazeer2023<empty citation> represent a forward dynamic model of the soft robotic arm as an RNN model that uses the two preceding states and three preceding control actions to predict the next state. The model is then used with the previously discussed MLP policy (section 3.1).

Lu2020<empty citation> was discussed in section 3.1 where it was briefly mentioned that training rollouts for the policies were generated using a dynamic model. This model is in fact

relevant for the basic idea of the policy.

also a data-driven model, being an ensemble of neural networks and predicting the next state s' based on the current state s and a control action yielded by the policy π_θ .

In and **Nguyen2011**<empty citation>, the inverse dynamic model is represented by a Gaussian process (a type of probabilistic model) and is learned through Local Gaussian Process Regression (**Nguyen2008a**; **Nguyen2009**). The models are employed in a feedforward control scheme, supplemented by a feedback term that adjusts the control action outputted by the dynamic model.

redundant references?

Romeres2016<empty citation> represent an inverse dynamic model of a robotic arm as a Gaussian process, but also experiment with using the arm's rigid body dynamics in several ways to enhance the model's performance. By incorporating the information from the robot's physical parameters into both the mean function and the covariance kernel of the Gaussian process, they develop (a) semi-parametric model(s) that are used in a feedforward manner where a feedback controller adds small corrections to the control actions outputted by the dynamic model.

Add paragraph on **Gijsberts2011**<emp

Contrary to blackbox policy learning methods, the adaptation to new tasks is not included in the learning process of the dynamic model, but is entirely dependent on the control policy. In case control policies have been defined in advance for several tasks, these task settings might serve as a test to verify that the dynamic model generalizes well over these different settings. In situations where tasks are not known in advance, a control scheme such as an incrementally updated (inverse) dynamics model in some form of a feedback control loop might serve for simple applications (**NguyenPeters2011**). For more complex tasks, a more intricate control system might be warranted.

rethink order of paragraphs?

3.3. Incremental learning

In section 3.1 and 3.2 an overview was provided of how blackbox machine learning techniques can be used to model either control policies or dynamic models (to be used in control policies). The choice of machine learning method can be important when it comes to the desired incremental nature of the learning techniques. To illustrate, consider these two main strategies in continually adaptive control:

1. All training is performed offline (after gathering sufficient data) and the robot is now completely capable of immediately adapting to changing dynamics it will encounter during its operational lifespan.
2. Training is performed incrementally as data comes in, which can potentially be done in real-time.

In both scenarios, the control of the robot would be continually adaptive. However, only strategy 2 aligns with the incremental objectives of the adaptive control challenge. In addition, the complex dynamics and many degrees of freedom of soft robots might make it difficult to accurately represent all possible degradations in dynamics (such as damages) or state-space configurations of the robot, potentially causing an imbalance in the training dataset of strategy 1. Incrementally learning to adapt the control policy or dynamic model is thus far more desirable, since it can cope with unforeseen circumstances.

Not all the previously discussed methods are - or have the possibility to be - implemented in an incremental manner. As an example, the two policies from **Zhou2022**<empty citation> mentioned in section 3.1 are trained sequentially: one online and the other offline. Each of these policies required different training/optimization algorithms for their specific setting, in this case Maximum a Posteriori Policy Optimization (MPO) (**Abdolmaleki2018**) for the on-line interaction setting, and Critic Regularized Regression (CRR) (**Wang2020**) for the offline distillation setting (see ??).

reference chapter

To learn the dynamic model from **Gijsberts2011**<empty citation>, the authors implement Ridge Regression (a linear regression method that aims at keeping the parameters as small as possible) incrementally, whereas normally it is used in a batch-learning setting. They achieve this by employing a modified numerical approach that is based on the QR algorithm (**Sayed2008**), a technique in linear algebra that can process new information incrementally using Givens rotations (a process that is particularly efficient for online learning scenarios) (**Bjorck1996numerical**). This enables real-time learning as new data comes in without recalculating the full model.

Lu2020<empty citation> implement both offline and online training, with the potential for the entire method to be implemented online. In fact, it is well known that SAC - the algorithm used for updating the skill-space distribution and the policy - is very suitable for online updates. When it comes to the dynamic model, they mention that is incrementally updated through interactions with the environment, but do not provide any details on the algorithm used for training the dynamic model.

In **Nguyen2011**<empty citation> the authors not only update their model incrementally, but they also manage to do it in real-time. This is achieved by using incremental Gaussian process regression algorithm, which uses only a select number of data points to fit a model. In addition, they introduce an efficient way to incrementally update the dictionary of datapoints in a way that it maintains the most informative points for a given budget. These two techniques are key to reducing the computational demand and allowing for real-time updates.

The semi-parametric models from **Romeres2016**<empty citation> are learned using the Recursive Least-Squares algorithm (**Landau2011**), which inherently updates the model incrementally. Much alike **Gijsberts2011**<empty citation>, this method uses a numerical trick from the field of adaptive filtering (Cholesky-based updates) to improve efficiency and enable updates in real-time.

3.4. Summary

In Table 3.1, a clear comparison of blackbox methods in lifelong learning is provided. This summary aims to give a straightforward reference, highlighting the strengths of blackbox methods and their role in lifelong learning.

Reference	Dynamic model	Control policy	Incremental
Gillespie2018 <empty citation>	SS-matrices	MPC	—
Gijsberts2011 <empty citation>	Linear model in random feature space	—	✓
Thuruthel2019 <empty citation> ²	RNN	Open-loop + trajectory optimization	—
Lu2020 <empty citation>	NN ensemble	MPPI + NN	✓
Nguyen2011 <empty citation>	Gaussian process	Feedforward	✓
Romeres2016 <empty citation>	Gaussian process	feedforward + feedback	✓
Nazeer2023 <empty citation>	RNN	MLP	³
Thuruthel2019 <empty citation> ⁴	—	FNN	—
Alessi2023 <empty citation>	—	FNN	—
Zhou2022 <empty citation>	—	Probabilistic model	✓

Table 3.1: An overview blackbox methods and their ability to be implemented incrementally. For dynamic model learning methods, the control policy specifies the control scheme in which the model is used.

²Used for generating training samples

³The incremental nature of the method is achieved by a third network, not by the dynamics model or control policy

⁴After training on generated samples

add Lu2019 to chapter, add the training algorithm to the table (eg. PPO)?

4

Learning architectures

Blackbox methods for lifelong learning are promising approaches to achieve continually adaptive control, but are hard-to-interpret. What other techniques can be implemented that are more intuitive in nature?

Well-designed control architectures can either enable or facilitate lifelong learning, whether the control schemes use blackbox models or more traditional methods. For example, some techniques use two or more blackbox models, where the idea is to have each model deal with a separate challenge of the lifelong learning goal. Other architectural methods aim to expand existing control schemes or policies in order to enable them to continually adapt.

In this chapter, these two main strategies will be further explored in the literature.

[1] Added introduction to chapter

4.1. Dual architectures

(Mallya2018; Schwarz2018; Shin2017; Lu2019; Lu2020; Sprechmann2018).

4.2. Architectural expansions

(Chatzilygeroudis2016; Nazeer2023).

5

Storage and retrieval

(Wang2022; Schulman2017; Mouret2015; Shin2017; Aljundi2016; Nguyen2011; Xie2021; Sprechmann2018).

6

Discussion and conclusion

Todo list

redundant references?	7
Add paragraph on Gijsberts2011<empty citation>	7
rethink order of paragraphs?	7
reference chapter	7
add Lu2019 to chapter, add the training algorithm to the table (eg. PPO)?	8

List of changes

Added: (Levine13)	5
Added: (Haarnoja2018)	6
Added: (Williams2015)	6
Added: (Sayed2008)	8
Added: (Bjorck1996numerical)	8
Added: (Landau2011)	8
Added: MPC	8
Commented: Added introduction to chapter	9